CHAPTER 27

Payment System

Move money correctly: pay-in, pay-out, idempotency, double-entry ledger and reconciliation.

Jump to: Problem statement | Clarifying questions to ask | Requirements | Back-of-the-envelope estimation | API / data model | High-level design | Deep dive | Trade-offs and alternatives | Scaling, reliability, failure modes | Likely follow-up questions | Common mistakes | Interview answer script | 60-second recap / practice checklist | References and further reading

1. Problem statement

Design a **payment backend** for an e-commerce platform (like Amazon/Shopify): customers pay (**pay-in**), the platform pays sellers (**pay-out**), using credit cards via external PSPs (Stripe, Adyen, PayPal). Prioritise **correctness, exactly-once effects, security, and auditability**. We do not build the card network.

Why interviewers ask it: Distributed-transaction thinking: idempotency, retries, PSP integration, ledger, reconciliation, failure handling.

2. Clarifying questions to ask (and sensible assumptions)

Question	Assumed answer
Which payments?	Credit card and other methods via third-party PSPs.
Handle card data ourselves?	No: use PSP-hosted payment page (PCI-DSS offloaded).
Scale?	1 M transactions/day.
Pay-in and pay-out?	Both.
Currencies / countries?	Multi-currency, global.
Fraud / compliance?	Required but not detailed (KYC/AML basics).

3. Requirements

Functional

- Pay-in: collect money from customers.
- Pay-out: pay sellers.
- Refunds and cancellations.
- Payment status lookup.
- Reconciliation with PSP/bank.

Non-functional

- **Reliability and fault tolerance:** payments must not be lost or duplicated.
- **Consistency** across payment service, ledger, wallet and PSP.
- Security: PCI-DSS, encryption, fraud checks.
- Auditability: immutable ledger.

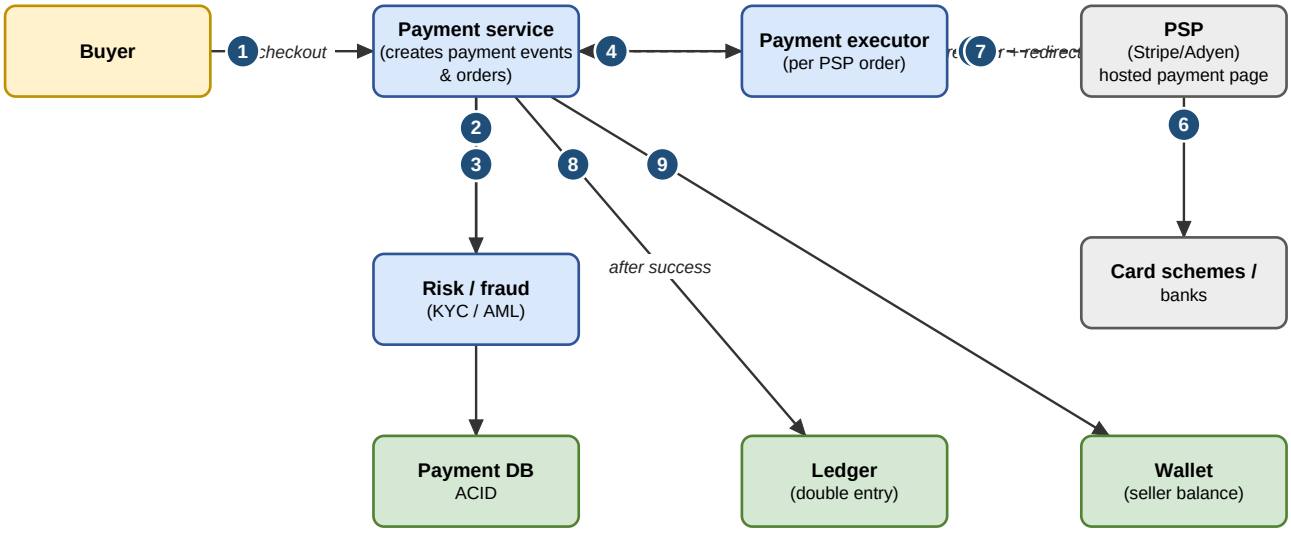
4. Back-of-the-envelope estimation

- 1 M transactions/day = about **10 TPS** average (peak 5-10x = 100 TPS). Low QPS: the focus is correctness, not throughput; use a relational DB with ACID.

5. API / data model

```
POST /v1/payments        { checkout_id, buyer_info, credit_card (token), amount, currency, payment_order_id }
GET  /v1/payments/{checkout_id}
Tables: payment_event(checkout_id, buyer, seller, amount, currency, is_payment_done), payment_order(id, seller_a
```

6. High-level design



Step	What happens
1	The buyer clicks **checkout**. The payment service receives a **payment event** with the order, and a unique **idempotency key** (e.g. the payment order ID).
2	The service runs risk, fraud and compliance checks (KYC / AML) before moving money.
3	It writes the payment event and a payment order (status `NOT_STARTED`) to the **payment DB** in an ACID transaction.
4	The **payment executor** takes each payment order, so one checkout with several sellers becomes several orders.
5	The executor calls the **PSP** (Stripe/Adyen) to register the payment, and the buyer is redirected to the PSP's **hosted payment page**, so card numbers never touch our servers (PCI scope).
6	The PSP talks to the card schemes and banks to authorize and capture.
7	The PSP sends a **webhook** with the result. Our payment service verifies and marks the order `SUCCESS` or `FAILED`; a reconciliation job covers lost webhooks.
8	On success the **ledger** records a **double-entry** record (debit buyer, credit seller); it is immutable and auditable.
9	The **wallet** updates the seller's balance.

Request flow

- User clicks Place Order -> the payment service stores a **payment event** and one **payment order** per seller (a checkout can contain several sellers) and a **nonce / token** for the PSP.
- The executor calls the PSP to register the payment; the user is redirected to the **PSP-hosted payment page** (card data never touches our servers).
- PSP processes and sends an async **webhook** with the result; the payment service updates the payment order status, then updates the **wallet** (seller balance) and **ledger** (double-entry) in order.
- Pay-out: periodically transfer seller balance via a PSP/ACH payout. Reconciliation compares our ledger with PSP settlement files.

7. Deep dive

Idempotency (exactly-once effect)

- Network retries or double clicks must not double-charge. Use an **idempotency key** (the `payment_order_id` generated by the server and passed to the PSP, or `Idempotency-Key` header). The DB has a unique constraint on that key; a retry returns the previous result.
- PSPs also accept the nonce/idempotency token so retries to the PSP are safe.

Retry policies and states

- Payment order states: NOT_STARTED -> EXECUTING -> SUCCESS / FAILED. Never mark SUCCESS until the PSP confirms. Use **exponential backoff** with limits; a **dead-letter queue** for payments needing manual review; distinguish retryable (timeout) from non-retryable (card declined) errors.
- A processing timeout is an unknown state - query the PSP status API (never blindly retry a charge without an idempotency key).

Double-entry ledger

- Every transaction creates two entries: **debit** the buyer account and **credit** the seller account with equal amounts; the sum of all entries is 0. **Immutable, append-only** records give auditability and let you detect inconsistencies. Wallet = derived balance.

Reconciliation

- Asynchronous nightly process: compare PSP settlement files (source of truth for external money) with our ledger; classify mismatches (classifiable & automatable, classifiable but manual, unclassifiable) and fix/escalate. This is the **last line of defence** for correctness.

Consistency across services

- Payment service, wallet, ledger can be separate databases -> avoid distributed transactions where possible: use **single-DB ACID** for wallet+ledger when feasible, or **Saga/TC-C** with compensations and an **outbox/event log** for reliable messaging.
- Use Kafka with transactional outbox for events; consumers must be idempotent.

Handling failures

- Payment executor failure: pick up from the persisted order state; webhook lost: periodic polling PSP; DB failure: replicas, backups; **exactly-once** emerges from at-least-once delivery + idempotent handlers.

Security and compliance

- Never store raw card numbers (PAN): use **tokenisation** and PSP-hosted pages (PCI scope reduction). TLS everywhere, encrypt at rest, secrets in HSM/KMS, HMAC-verify webhooks, **fraud detection** (rules + ML), rate limiting, audit logs, KYC/AML for payouts.

Currency and rounding

- Store amounts as integers in minor units (cents) with ISO currency; never floats. Keep FX rate snapshots with each transaction.

8. Trade-offs and alternatives

Decision	Option A / Option B	How to choose
Card handling	Own card storage / PSP-hosted page	PSP-hosted: PCI scope reduced, safer.
Consistency	Distributed transaction (2PC) / Saga + idempotency	Prefer ACID within a service; Saga + reconciliation across services.
Retries	At-most-once / at-least-once + idempotency	At-least-once + idempotent keys is the standard.
Status updates	Webhook only / webhook + polling	Use both; polling covers lost callbacks.

9. Scaling, reliability, failure modes

- Low TPS but must scale for sales events: stateless services, queue-buffered executors, sharded DB by merchant/order.
- Active-passive DR with synchronous replication for the ledger; regular failover drills.
- Strict monitoring of success rates per PSP and alerting.

10. Likely follow-up questions

- What happens if the PSP times out after charging the card?
- How do you prevent double refunds?
- How do you design pay-out batching and failure handling?
- How do you support multiple PSPs and routing/failover between them?

11. Common mistakes

- Using floating-point numbers for money.
- Treating a timeout as failure and charging again.
- Skipping reconciliation.
- Storing card details yourself.

12. Interview answer script (what to say, in order)

Follow the 4-step framework from chapter 04. Say it out loud; the quoted text is a model answer in your own voice.

Phase	Time	What to say
1. Understand + scope	5 min	"Scope: pay-in from buyers to the platform and pay-out to sellers, using a PSP for card processing. 1 M transactions a day is only about 10 TPS, so correctness beats throughput. Requirements: no double charge, no lost payment, auditability, reconciliation and security. I'd confirm whether we store cards (no, the PSP does)."
2. High-level design	10 min	"A payment service records payment events and orders in an ACID payment DB, calls the executor, which registers with the PSP and redirects the buyer to the hosted page. The PSP calls back via webhook. On success we record a double-entry ledger and update the wallet. Fraud checks happen in the flow."
3. Deep dive	25 min	"The key topics are reliability and consistency. I make every payment idempotent with an idempotency key, so a retry doesn't double charge. I keep a state machine for each payment order. Because exactly-once is impossible, I use at-least-once with retries plus idempotent handlers. Reconciliation compares our records with the PSP's settlement files every day and flags mismatches. For failures I'd use a retry queue and dead-letter queue. For security: HTTPS, tokenization, hosted page, signed webhooks."
4. Wrap up	4 min	"Extras: multi-currency, chargebacks and refunds, PSP failover, and monitoring success rate per PSP. Sensitive data is encrypted and access is audited."

13. 60-second recap / practice checklist

- Pay-in flow with PSP-hosted page + webhook; pay-out flow with batching.
- Idempotency key, retry/DLQ, immutable double-entry ledger, reconciliation.
- Security (tokenisation, PCI, fraud), correctness over throughput.

14. References and further reading

Cross-checked against these sources (links verified when this PDF was built).

• Stripe: Idempotency
• Square: Books - immutable double-entry accounting
• PCI Security Standards Council